LambdaScript : A Functional Programming Language

Alex Kozik
Cornell University
Ithaca, NY
ajk333@cornell.edu

Abstract

We present LambdaScript, a statically-typed functional programming language inspired by Haskell and OCaml. LambdaScript features Hindley-Milner type inference, polymorphic algebraic data types, comprehensive pattern matching, and first-class functions with closures. The language supports advanced features including mutually recursive type definitions, list comprehensions, and operators as first-class values. This paper describes both the formal semantics and the complete implementation of LambdaScript, including the design of the type system, operational semantics, and the architecture of a full interpreter pipeline consisting of lexical analysis, parser combinators, constraint-based type inference, and environment-based evaluation. The implementation is validated through 1200 unit tests covering complex real-world algorithms including red-black trees, interpreters, and graph traversal.

Index Terms

functional programming, type inference, algebraic data types, pattern matching, programming languages, formal semantics

CONTENTS

I Introduction 3

II Examples 4
- II-A Polymorphic List Data Type . . . 4
  - II-A1 Map Function . . . 4
  - II-A2 Reduce Function . . . 4
- II-B Option Monad . . . 4
  - II-B1 Monadic Bind Operation . . . 4
  - II-B2 Return Operation . . . 5
  - II-B3 Map for Option . . . 5
  - II-B4 Example: Safe Division . . . 5

III Type System 6
- III-A Basic Types and Type Inference . . . 6
- III-B Type Variables and Generalization . . . 6
- III-C Algebraic Data Types . . . 6
- III-D Recursive Types . . . 6
- III-E Mutually Recursive Types . . . 7
- III-F Type Annotations . . . 7

IV Syntax 8

V Formal Semantics 10
- V-A Overview . . . 10
- V-B Static Semantics . . . 10
  - V-B1 Notation and Definitions . . . 10
  - V-B2 Typing Rules . . . 11
  - V-B3 Pattern Typing . . . 12
  - V-B4 Constraint-Based Type Inference . . . 13
- V-C Dynamic Semantics . . . 13
  - V-C1 Notation . . . 13
  - V-C2 Evaluation Rules . . . 14
  - V-C3 Pattern Matching Rules . . . 15
  - V-C4 Evaluation Examples . . . 16

VI Implementation 17
VI-A Architecture Overview . . . 17
VI-B Lexer and Parser . . . 17
VI-B1 Lexical Analysis . . . 17
VI-B2 Parser Combinators . . . 17
VI-C Type Inference Engine . . . 18
VI-C1 Constraint Generation . . . 18
VI-C2 Unification . . . 18
VI-C3 Generalization and Instantiation . . . 18
VI-C4 Type Fixing . . . 19
VI-D Pattern Matching . . . 19
VI-D1 Pattern Compilation . . . 19
VI-D2 Exhaustiveness Checking . . . 19
VI-E Evaluator . . . 19
VI-E1 Value Representation . . . 19
VI-E2 Environment-Based Evaluation . . . 19
VI-E3 Built-in Functions . . . 20
VI-F Advanced Features . . . 20
VI-F1 Mutually Recursive Types . . . 20
VI-F2 List Comprehensions . . . 20
VI-F3 Custom Operators . . . 20
VI-F4 Code Blocks . . . 20

VII Testing and Validation 21
VII-A Test Suite Design . . . 21
VII-B Coverage Analysis . . . 21
VII-C Real-World Examples . . . 21

VIII Discussion 22
VIII-A Known Limitations . . . 22
VIII-B Related Work . . . 22
VIII-C Future Work . . . 22

IX Conclusion 23

I. Introduction

Statically-typed functional programming languages such as Haskell, OCaml, and Standard ML provide powerful abstractions for writing correct programs through algebraic data types, pattern matching, and polymorphic type inference. The Hindley-Milner type system enables programmers to write code without explicit type annotations while maintaining complete type safety through automatic type inference. However, understanding how these features interact—both formally through typing rules and operationally through implementation—requires navigating complex, mature language ecosystems with decades of accumulated features and optimizations.

This paper presents **LambdaScript**, a functional programming language designed to demonstrate that rigorous formal semantics and practical implementation can coexist harmoniously. LambdaScript features a complete Hindley-Milner type inference system, polymorphic algebraic data types, comprehensive pattern matching, and advanced features including mutually recursive type definitions. Rather than merely describing theoretical properties, we provide both a complete formal semantics and a fully implemented interpreter validated through extensive testing.

A key contribution of LambdaScript is its treatment of **mutually recursive algebraic data types**, which enable natural representation of complex data structures such as trees with forests, abstract syntax trees with multiple node types, and graph-based structures. While languages like OCaml and Haskell support such types, their formal semantics and implementation present subtle challenges in constraint generation, type environment management, and ensuring that recursive references are properly resolved during type checking. LambdaScript addresses these challenges with a clean syntax using the `and` keyword and a carefully designed type checking algorithm that extends the standard Hindley-Milner approach.

The implementation consists of a complete interpreter pipeline: a lexer that tokenizes source code, a parser built using parser combinators for compositional grammar definitions, a constraint-based type checker implementing Hindley-Milner inference with extensions for recursive types, and an environment-based evaluator that handles closures, recursion, and pattern matching. The entire system is written in OCaml and validated through 1200 unit tests covering type inference edge cases, evaluation correctness, and real-world algorithms including red-black tree operations.

The main contributions of this paper are:

- A formal semantics for a functional language with mutually recursive algebraic data types, including complete typing rules and operational semantics
- A complete implementation architecture demonstrating how theory translates to practice, including lexer, parser combinators, constraint-based type inference, and environment-based evaluation
- Extensions to the standard Hindley-Milner algorithm to support mutually recursive types and proper handling of fixed-point types
- Validation through 1200 unit tests demonstrating correctness on complex algorithms including red-black trees, showcasing the language's expressiveness

The remainder of this paper is organized as follows. Section II presents examples demonstrating LambdaScript's features, building intuition before formal treatment. Section III describes the type system design, including polymorphism, algebraic data types, and type inference. Section IV provides detailed formal semantics with typing rules and operational semantics. Section V details the implementation architecture of each pipeline component. Section VI presents testing methodology and evaluation results. Section VII concludes with reflections and future directions.

II. Examples

This section presents examples that demonstrate LambdaScript's core features, building intuition for the formal treatment in subsequent sections. We showcase algebraic data types, pattern matching, polymorphism, and higher-order functions through practical examples.

A. Polymorphic List Data Type

We begin by defining a polymorphic list type using recursive algebraic data types. LambdaScript uses the `type rec` keyword for recursive type definitions and the `'a` syntax for type variables, following OCaml conventions.

```
type rec List<'a> =
  | Nil
  | Cons of ('a, List<'a>)
```

This definition introduces two constructors: `Nil` represents the empty list, and `Cons` takes a tuple containing an element of type `'a` and the rest of the list. The type parameter `'a` makes this list polymorphic, allowing lists of any type.

With the list type defined, we can implement fundamental higher-order functions that operate over lists.

1) Map Function: The `map` function applies a function to each element of a list:

```
let rec map f lst =
  case lst do
  | Nil -> Nil
  | Cons (h, t) -> Cons (f h, map f t)
```

The type system infers the polymorphic type:

```
map : ('a -> 'b) -> List<'a> -> List<'b>
```

This demonstrates parametric polymorphism: `map` works for any input type `'a` and output type `'b`, as long as the function `f` has type `'a -> 'b`.

Example usage:

```
let double = fn x -> x * 2 in
let numbers = Cons (1, Cons (2, Nil)) in
map double numbers
(* Result: Cons (2, Cons (4, Nil)) *)
```

2) Reduce Function: The `reduce` function (also known as fold) combines list elements using a binary operator:

```
let rec reduce op acc lst =
  case lst do
  | Nil -> acc
  | Cons (h, t) -> reduce op (op acc h) t
```

Inferred type: `reduce : ('b -> 'a -> 'b) -> 'b -> List<'a> -> 'b`

The type signature shows that `reduce` can produce a result of a different type (`'b`) than the list elements (`'a`), demonstrating the power of polymorphic type inference.

Example summing a list:

```
let add = fn x -> fn y -> x + y in
let numbers = Cons (1, Cons (2, Nil)) in
reduce add 0 numbers
(* Result: 3 *)
```

B. Option Monad

The Option type represents computations that may fail, providing a type-safe alternative to null values. We define it as a sum type with two constructors:

```
type Option<'a> =
  | None
  | Some of 'a
```

`None` represents absence of a value, while `Some` wraps a present value of type `'a`.

1) Monadic Bind Operation: The bind operation chains optional computations. In LambdaScript, we can define it as an operator:

```
let (>>=) opt f =
  case opt do
  | None -> None
  | Some x -> f x
```

Inferred type: `(>>=) : Option<'a> -> ('a -> Option<'b>) -> Option<'b>`

This type signature is the hallmark of monadic bind: it takes an `Option<'a>`, a function from `'a` to `Option<'b>`, and produces `Option<'b>`. If the input is `None`, the computation short-circuits; otherwise, the function is applied to the unwrapped value.

2) Return Operation: The `return` operation (called `pure` in some languages) lifts a value into the Option context:

```
let return x = Some x
```

Inferred type: `return : 'a -> Option<'a>`

3) Map for Option: Mapping a function over an optional value:

```
let map_opt f opt =
  case opt do
  | None -> None
  | Some x -> Some (f x)
```

Inferred type: `map_opt : ('a -> 'b) -> Option<'a> -> Option<'b>`

4) Example: Safe Division: These operations enable safe computation chains. Consider safe division:

```
let safe_div x y =
  if y == 0 then None
  else Some (x / y)
in

let compute x y z =
  (safe_div x y) >>= (fn result1 ->
    (safe_div result1 z) >>= (fn result2 ->
      return result2))
in

compute 100 10 2
(* Result: Some 5 *)

compute 100 0 2
(* Result: None *)
```

The first computation succeeds: $100/10/2 = 5$. The second short-circuits at division by zero, returning `None` without attempting the second division. This demonstrates how the Option monad handles errors compositionally through pattern matching and type safety, without exceptions or null pointer errors.

These examples illustrate several key features of LambdaScript: algebraic data types enable clean data modeling, pattern matching provides exhaustive case analysis, parametric polymorphism allows generic code, and the type system infers complex types including higher-rank polymorphism in monadic operations.

III. Type System

LambdaScript features a Hindley-Milner type system with support for parametric polymorphism, algebraic data types, and mutually recursive type definitions. The type system is both sound and complete: every well-typed program has a principal type, and type inference always succeeds when a valid typing exists.

A. Basic Types and Type Inference

LambdaScript includes primitive types (`int`, `float`, `bool`, `string`, `char`, `unit`), composite types (functions, lists, tuples), and user-defined algebraic data types. The type checker automatically infers types without requiring explicit annotations:

```
let x = 42
(* Inferred: x : int *)

let add x y = x + y
(* Inferred: add : int -> int -> int *)

let id x = x
(* Inferred: id : 'a -> 'a *)
```

The identity function `id` receives the polymorphic type `'a -> 'a`, where `'a` is a type variable that can be instantiated to any type. This demonstrates **let-polymorphism**: functions bound with `let` can be used at multiple types in different contexts.

B. Type Variables and Generalization

Type variables represent unknown types during inference. When a function is defined with `let`, the type checker **generalizes** over all unconstrained type variables, introducing universal quantifiers. Consider:

```
let compose f g x = f (g x)
(* Type: ('b -> 'c) -> ('a -> 'b) -> 'a -> 'c *)
```

The type checker generates fresh type variables for `f`, `g`, and `x`, then analyzes the function application `f (g x)`:

- `g x` requires `g : 'a -> 'b` for some `'b`
- `f (g x)` requires `f : 'b -> 'c` for some `'c`
- The result has type `'c`

After constraint solving, the system generalizes to produce the most general type with three independent type variables.

C. Algebraic Data Types

Algebraic data types define new types using constructors. Each constructor may carry a payload:

```
type Result<'a, 'b> =
  | Ok of 'a
  | Error of 'b
```

When this definition is processed, the type checker:

1) Introduces `Result` as a new type constructor taking two type parameters
2) Creates constructor types: `Ok : 'a -> Result<'a, 'b>` and `Error : 'b -> Result<'a, 'b>`
3) Adds these constructors to the typing environment

Users can then construct and match on values:

```
let safe_sqrt x =
  if x < 0 then Error "negative"
  else Ok (sqrt x)
(* Type: int -> Result<float, string> *)

let handle result =
  case result do
  | Ok v -> v
  | Error msg -> 0.0
(* Type: Result<float, 'a> -> float *)
```

D. Recursive Types

Recursive types reference themselves in their definition. LambdaScript uses **isorecursive** semantics with explicit `type rec` declarations:

```
type rec Tree<'a> =
  | Leaf of 'a
  | Node of ('a, Tree<'a>, Tree<'a>)
```

Internally, this is represented as a fixed-point type μ`Tree`. The type checker ensures that:

- Each recursive occurrence is properly parameterized
- Constructor payloads are type-checked in an environment containing the recursive type
- Pattern matching correctly unfolds the recursive type

Example usage demonstrates automatic type inference through recursive structures:

```
let rec height tree =
  case tree do
  | Leaf _ -> 1
  | Node (_, l, r) ->
      1 + (if height l > height r
           then height l else height r)
(* Type: Tree<'a> -> int *)
```

E. Mutually Recursive Types

A key contribution of LambdaScript is clean support for mutually recursive type definitions using the `and` keyword. Multiple types can reference each other:

```
type rec Tree<'a> =
  | Leaf of 'a
  | Node of ('a, Forest<'a>)
and Forest<'a> =
  | Empty
  | Trees of (Tree<'a>, Forest<'a>)
```

The type checker processes mutually recursive types by:

1) Adding all type names to the environment simultaneously before processing any constructors
2) Type-checking each constructor's payload in the extended environment
3) Ensuring all type parameters are consistently applied across mutual references
4) Verifying that cycles are properly guarded by constructors (no infinite types)

This enables natural expression of complex data structures:

```
let rec tree_size t =
  case t do
  | Leaf _ -> 1
  | Node (_, forest) -> 1 + forest_size forest
and forest_size f =
  case f do
  | Empty -> 0
  | Trees (t, rest) ->
      tree_size t + forest_size rest
(* tree_size : Tree<'a> -> int
   forest_size : Forest<'a> -> int *)
```

The implementation extends the standard Hindley-Milner unification algorithm to handle fixed-point types and ensures that type variable substitutions propagate correctly across the mutual recursion group.

F. Type Annotations

While type inference eliminates most annotations, LambdaScript allows optional type annotations for documentation and disambiguation:

```
let f (x: int) : int = x + 1

let g (x: int) : int = x * 2
```

Type annotations are checked for consistency with inferred types. If an annotation conflicts with the inferred type, the type checker reports an error.

IV. Syntax

We present the complete syntax of LambdaScript in Extended Backus-Naur Form (EBNF). Non-terminals are written in italics, terminals in `typewriter font`, and metavariables are defined below.

Expressions:

$$
\begin{aligned}
e ::= {} & n \mid b \mid s \mid c \mid () \mid x \\
& \mid e_1 \ \texttt{op} \ e_2 \\
& \mid \texttt{fn} \ p \ \texttt{->} \ e \\
& \mid e_1 \ e_2 \\
& \mid \texttt{let} \ p = e_1 \ \texttt{in} \ e_2 \\
& \mid \texttt{let rec} \ x \ p^* = e_1 \ \texttt{in} \ e_2 \\
& \mid \texttt{if} \ e_1 \ \texttt{then} \ e_2 \ \texttt{else} \ e_3 \\
& \mid \texttt{case} \ e \ \texttt{do} \ (\texttt{|} \ p \ \texttt{->} \ e)^+ \\
& \mid [\,] \mid e_1 \ \texttt{::} \ e_2 \mid [\, e_1, \ldots, e_n \,] \\
& \mid (e_1, \ldots, e_n) \\
& \mid C \mid C \ e
\end{aligned}
$$

Patterns:

$$
\begin{aligned}
p ::= {} & \_ \mid x \mid n \mid b \mid s \mid c \mid () \\
& \mid [\,] \mid p_1 \ \texttt{::} \ p_2 \\
& \mid (p_1, \ldots, p_n) \\
& \mid C \mid C \ p
\end{aligned}
$$

Types:

$$
\begin{aligned}
\tau ::= {} & \texttt{int} \mid \texttt{float} \mid \texttt{bool} \mid \texttt{string} \mid \texttt{char} \mid \texttt{unit} \\
& \mid \alpha \mid \tau_1 \ \texttt{->} \ \tau_2 \mid [\, \tau \,] \\
& \mid (\tau_1, \ldots, \tau_n) \\
& \mid T \mid T \ \texttt{<}\tau_1, \ldots, \tau_n\texttt{>}
\end{aligned}
$$

Type Definitions:

$$
\begin{aligned}
\textit{typedef} ::= {} & \texttt{type} \ T \ \texttt{<}\alpha_1, \ldots, \alpha_n\texttt{>} = \tau \\
& \mid \texttt{type} \ T \ \texttt{<}\alpha_1, \ldots, \alpha_n\texttt{>} = (\texttt{|} \ C \ [\texttt{of} \ \tau])^+ \\
& \mid \texttt{type rec} \ T \ \texttt{<}\alpha_1, \ldots, \alpha_n\texttt{>} = (\texttt{|} \ C \ [\texttt{of} \ \tau])^+ \\
& \mid \texttt{type rec} \ T_1 \ \texttt{<}\alpha^*\texttt{>} = (\texttt{|} \ C \ [\texttt{of} \ \tau])^+ \\
& \ \ (\texttt{and} \ T_i \ \texttt{<}\alpha^*\texttt{>} = (\texttt{|} \ C \ [\texttt{of} \ \tau])^+)^+
\end{aligned}
$$

Programs:

$$
\textit{program} ::= (\textit{typedef} \mid \texttt{let} \ p = e \mid \texttt{let rec} \ x \ p^* = e)^*
$$

Metavariables:

- x ranges over variables (identifiers beginning with lowercase letters)
- n ranges over integer literals
- b ranges over boolean literals (`true`, `false`)
- s ranges over string literals
- c ranges over character literals
- C ranges over constructors (identifiers beginning with uppercase letters)
- T ranges over type constructors
- α ranges over type variables (identifiers beginning with `'`)
- `op` ranges over binary operators: `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `&&`, `||`

Notation:

- e^* denotes zero or more occurrences of e
- e^+ denotes one or more occurrences of e
- $[e]$ denotes an optional occurrence of e
- $(e_1 \mid e_2)$ denotes a choice between e_1 and e_2

V. Formal Semantics

A. Overview

We now present the formal semantics of LambdaScript, providing a rigorous mathematical foundation for both the type system and runtime behavior. Formal semantics serve multiple purposes: they enable precise reasoning about program behavior, provide unambiguous specifications for implementers, and allow formal verification of type soundness and other properties.

Our semantic framework consists of two complementary components:

Static Semantics defines the type system through typing judgments of the form $\Gamma \vdash e : \tau$, read as "under type environment Γ, expression e has type τ." The static semantics specify when programs are well-typed and determine the principal types of expressions through constraint-based type inference. We formalize the Hindley-Milner algorithm, including constraint generation, unification, and generalization. Special attention is given to the treatment of algebraic data types and mutually recursive type definitions, which require careful handling of type environments and fixed-point types.

Dynamic Semantics defines program evaluation through operational semantics. We use large-step (natural) operational semantics with evaluation judgments of the form $\langle e, \rho \rangle \Downarrow v$, indicating that expression e in environment ρ evaluates to value v. Large-step semantics directly relate expressions to their final values, making the evaluation rules more intuitive and closely aligned with the implementation. The dynamic semantics specify how values are computed, how functions are applied, how pattern matching proceeds, and how recursion is handled through environment-based closure semantics.

The static and dynamic semantics are designed to satisfy **type soundness**: well-typed programs do not get stuck during evaluation. This is formalized through the type preservation theorem: if $\Gamma \vdash e : \tau$ and $\langle e, \rho \rangle \Downarrow v$, then v has type τ. While we do not provide complete proofs here, the semantics are structured to facilitate such proofs.

We adopt standard mathematical notation: Γ denotes type environments mapping variables to types, ρ denotes value environments mapping variables to values, and τ ranges over types. Substitutions are written $[\tau'/\alpha]\tau$ for replacing type variable α with type τ' in type τ. Function types are written $\tau_1 \to \tau_2$, list types as $[\tau]$, and tuple types as $(\tau_1, \tau_2, \ldots, \tau_n)$.

B. Static Semantics

The static semantics of LambdaScript defines a type system based on the Hindley-Milner approach with extensions for algebraic data types and mutually recursive type definitions. We present typing rules, constraint generation, unification, and generalization.

1) Notation and Definitions: **Types and Type Schemes:**

Types τ are classified as:

- Primitive types: `int`, `bool`, `string`, `char`, `unit`
- Type variables: $\alpha, \beta, \gamma, \ldots$
- Function types: $\tau_1 \to \tau_2$
- List types: $[\tau]$
- Tuple types: $(\tau_1, \ldots, \tau_n)$
- Algebraic data types: $T\langle\tau_1, \ldots, \tau_n\rangle$
- Recursive types: $\mu T.\tau$

Type schemes σ extend types with universal quantification:

$$\sigma ::= \tau \mid \forall \alpha.\sigma$$

Type Environments:

A type environment Γ is a finite map from variables to type schemes, written $\{x_1 : \sigma_1, \ldots, x_n : \sigma_n\}$. We write:

- $\Gamma(x)$ for environment lookup
- $\Gamma, x : \sigma$ for environment extension
- $\emptyset$ for the empty environment
- $FV(\Gamma)$ for the set of free type variables in Γ
- $FV(\tau)$ for the set of free type variables in τ

Substitutions:

A type substitution S is a finite map from type variables to types, written $[\tau_1/\alpha_1, \ldots, \tau_n/\alpha_n]$. We write:

- $S(\tau)$ for applying substitution S to type τ
- $S(\Gamma)$ for applying substitution to all types in Γ
- $S_2 \circ S_1$ for composition: $(S_2 \circ S_1)(\tau) = S_2(S_1(\tau))$
- `id` for the identity substitution

Typing Judgment:

The primary judgment form is:

$$\Gamma \vdash e : \tau$$

read as "under type environment Γ, expression e has type τ."

Generalization and Instantiation:

Generalization introduces universal quantifiers:

$$\text{gen}(\Gamma, \tau) = \forall\{\alpha_1, \ldots, \alpha_n\}.\tau$$

where $\{\alpha_1, \ldots, \alpha_n\} = FV(\tau) \setminus FV(\Gamma)$.

Instantiation removes universal quantifiers by substituting fresh type variables:

$$\text{inst}(\forall\alpha.\sigma) = \sigma[\beta/\alpha]$$

where β is fresh. For non-quantified types: $\text{inst}(\tau) = \tau$.

2) Typing Rules: We present the typing rules for all LambdaScript expressions.

Literals:

$$\frac{}{\Gamma \vdash n : \texttt{int}} \quad \frac{}{\Gamma \vdash b : \texttt{bool}} \quad \frac{}{\Gamma \vdash s : \texttt{string}} \quad \frac{}{\Gamma \vdash c : \texttt{char}} \quad \frac{}{\Gamma \vdash () : \texttt{unit}}$$

Variables:

$$\frac{x : \sigma \in \Gamma \quad \tau = \text{inst}(\sigma)}{\Gamma \vdash x : \tau}$$

Variables are looked up in the environment and their type schemes are instantiated with fresh type variables.

Functions:

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \texttt{fn}\ x\ \texttt{->}\ e : \tau_1 \to \tau_2}$$

For pattern-based functions:

$$\frac{p : \tau_1 \rightsquigarrow \Gamma' \quad \Gamma, \Gamma' \vdash e : \tau_2}{\Gamma \vdash \texttt{fn}\ p\ \texttt{->}\ e : \tau_1 \to \tau_2}$$

where $p : \tau \rightsquigarrow \Gamma'$ means pattern p matches values of type τ and produces bindings Γ'.

Application:

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$

Let Binding (Polymorphic):

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \text{gen}(\Gamma, \tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}$$

Let-bound variables are generalized, allowing polymorphic usage in e_2.

Recursive Let Binding:

$$\frac{\Gamma, f : \tau_1 \to \tau_2 \vdash \texttt{fn}\ x\ \texttt{->}\ e_1 : \tau_1 \to \tau_2 \quad \Gamma, f : \forall\{\alpha\}.(\tau_1 \to \tau_2) \vdash e_2 : \tau}{\Gamma \vdash \texttt{let rec}\ f\ x = e_1\ \texttt{in}\ e_2 : \tau}$$

The recursive function f is available in its own body with a monomorphic type, then generalized for use in e_2.

Conditional:

$$\frac{\Gamma \vdash e_1 : \texttt{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau}$$

Both branches must have the same type.

Pattern Matching:

$$\frac{\Gamma \vdash e : \tau \quad \forall i.\ p_i : \tau \rightsquigarrow \Gamma_i \quad \Gamma, \Gamma_i \vdash e_i : \tau'}{\Gamma \vdash \texttt{case}\ e\ \texttt{do}\ \{p_i \to e_i\}_{i=1}^{n} : \tau'}$$

All patterns must match the scrutinee type, and all branches must have the same result type.

Lists:

$$\frac{}{\Gamma \vdash [] : [\alpha]} \qquad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : [\tau]}{\Gamma \vdash e_1 :: e_2 : [\tau]}$$

Empty lists are polymorphic; cons requires consistent element types.

Tuples:

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \cdots \quad \Gamma \vdash e_n : \tau_n}{\Gamma \vdash (e_1, \ldots, e_n) : (\tau_1, \ldots, \tau_n)}$$

Constructors:

For a constructor $C : \tau \to T\langle \alpha_1, \ldots, \alpha_n \rangle$ in the environment:

$$\frac{}{\Gamma \vdash C : \tau[\tau_1/\alpha_1, \ldots, \tau_n/\alpha_n] \to T\langle \tau_1, \ldots, \tau_n \rangle}$$

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash C\ e : T\langle \ldots \rangle}$$

Constructors are instantiated with fresh type variables when used.

Binary Operators:

$$\frac{\Gamma \vdash e_1 : \texttt{int} \quad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1 \texttt{ + } e_2 : \texttt{int}}$$

Similar rules apply for other arithmetic operators ($\texttt{-}$, $\texttt{*}$, $\texttt{/}$, $\texttt{\%}$).

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 \texttt{ == } e_2 : \texttt{bool}}$$

Comparison operators require both operands to have the same type.

$$\frac{\Gamma \vdash e_1 : \texttt{bool} \quad \Gamma \vdash e_2 : \texttt{bool}}{\Gamma \vdash e_1 \texttt{ \&\& } e_2 : \texttt{bool}}$$

Logical operators require boolean operands.

3) Pattern Typing: Pattern typing judgments have the form $p : \tau \rightsquigarrow \Gamma$, indicating that pattern p matches values of type τ and produces bindings Γ.

Variable Pattern:

$$x : \tau \rightsquigarrow \{x : \tau\}$$

Wildcard Pattern:

$$\_ : \tau \rightsquigarrow \emptyset$$

Literal Patterns:

$$n : \texttt{int} \rightsquigarrow \emptyset \quad b : \texttt{bool} \rightsquigarrow \emptyset \quad () : \texttt{unit} \rightsquigarrow \emptyset$$

List Patterns:

$$[] : [\alpha] \rightsquigarrow \emptyset$$

$$\frac{p_1 : \tau \rightsquigarrow \Gamma_1 \quad p_2 : [\tau] \rightsquigarrow \Gamma_2}{p_1 :: p_2 : [\tau] \rightsquigarrow \Gamma_1 \cup \Gamma_2}$$

Tuple Patterns:

$$\frac{p_1 : \tau_1 \rightsquigarrow \Gamma_1 \quad \cdots \quad p_n : \tau_n \rightsquigarrow \Gamma_n}{(p_1, \ldots, p_n) : (\tau_1, \ldots, \tau_n) \rightsquigarrow \Gamma_1 \cup \cdots \cup \Gamma_n}$$

Constructor Patterns:

$$C : T\langle \tau_1, \ldots, \tau_n \rangle \rightsquigarrow \emptyset$$

$$\frac{p : \tau \rightsquigarrow \Gamma}{C\ p : T\langle \ldots \rangle \rightsquigarrow \Gamma}$$

where C has type $\tau \to T\langle \ldots \rangle$.

4) Constraint-Based Type Inference: LambdaScript implements type inference using constraint generation and unification, following Algorithm W.

Constraint Generation:

The function $\mathcal{C}(\Gamma, e)$ generates a pair (τ, C) where τ is the inferred type (possibly containing fresh type variables) and C is a set of type equality constraints.

Examples:

$$\begin{aligned}\mathcal{C}(\Gamma, n) &= (\texttt{int}, \emptyset)\\ \mathcal{C}(\Gamma, x) &= (\text{inst}(\Gamma(x)), \emptyset)\\ \mathcal{C}(\Gamma, e_1\ e_2) &= (\beta, C_1 \cup C_2 \cup \{(\tau_1, \tau_2 \to \beta)\})\end{aligned}$$

where $(\tau_1, C_1) = \mathcal{C}(\Gamma, e_1)$, $(\tau_2, C_2) = \mathcal{C}(\Gamma, e_2)$, and β is fresh.

$$\mathcal{C}(\Gamma, \texttt{fn}\ x\ \texttt{->}\ e) = (\alpha \to \tau, C)$$

where α is fresh, $(\tau, C) = \mathcal{C}(\Gamma \cup \{x : \alpha\}, e)$.

$$\mathcal{C}(\Gamma, \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3) = (\tau_2, C_1 \cup C_2 \cup C_3 \cup \{(\tau_1, \texttt{bool}), (\tau_2, \tau_3)\})$$

where $(\tau_i, C_i) = \mathcal{C}(\Gamma, e_i)$.

Unification:

The unification algorithm $\mathcal{U}(C)$ takes a set of constraints and computes a most general unifier (MGU), a substitution S satisfying all constraints.

Unification rules:

$$\begin{aligned}\mathcal{U}(\emptyset) &= \texttt{id}\\ \mathcal{U}(\{(\tau, \tau)\} \cup C) &= \mathcal{U}(C)\\ \mathcal{U}(\{(\alpha, \tau)\} \cup C) &= [\tau/\alpha] \circ \mathcal{U}([\tau/\alpha]C) \quad \text{if } \alpha \notin FV(\tau)\\ \mathcal{U}(\{(\tau, \alpha)\} \cup C) &= [\tau/\alpha] \circ \mathcal{U}([\tau/\alpha]C) \quad \text{if } \alpha \notin FV(\tau)\\ \mathcal{U}(\{(\tau_1 \to \tau_2, \tau_1' \to \tau_2')\} \cup C) &= \mathcal{U}(\{(\tau_1, \tau_1'), (\tau_2, \tau_2')\} \cup C)\\ \mathcal{U}(\{([\tau], [\tau'])\} \cup C) &= \mathcal{U}(\{(\tau, \tau')\} \cup C)\\ \mathcal{U}(\{((\tau_1, \ldots, \tau_n), (\tau_1', \ldots, \tau_n'))\} \cup C) &= \mathcal{U}(\{(\tau_1, \tau_1'), \ldots, (\tau_n, \tau_n')\} \cup C)\\ \mathcal{U}(\{(T\langle\tau_1, \ldots, \tau_n\rangle, T\langle\tau_1', \ldots, \tau_n'\rangle)\} \cup C) &= \mathcal{U}(\{(\tau_1, \tau_1'), \ldots, (\tau_n, \tau_n')\} \cup C)\end{aligned}$$

The occurs check ($\alpha \notin FV(\tau)$) prevents infinite types.

Type Inference Algorithm:

The complete type inference algorithm for expression e in environment Γ:

1) Generate constraints: $(\tau, C) = \mathcal{C}(\Gamma, e)$
2) Solve constraints: $S = \mathcal{U}(C)$
3) Apply substitution: $\tau' = S(\tau)$
4) Generalize: $\sigma = \text{gen}(S(\Gamma), \tau')$

The result is the principal type scheme σ.

Soundness and Completeness:

The type inference algorithm is sound (if inference succeeds, the program is well-typed) and complete (if a typing exists, inference finds the principal type). This is guaranteed by the properties of Algorithm W for Hindley-Milner type systems.

C. Dynamic Semantics

1) Notation: We employ the following notation for dynamic semantics:

Syntactic Categories:

- e ranges over expressions
- v ranges over values
- p ranges over patterns
- x ranges over variables

- n ranges over integer literals
- b ranges over boolean literals (`true`, `false`)
- C ranges over constructors

Values: Values are the results of evaluation and include:

- Integer values: n
- Boolean values: b
- Unit value: $()$
- String values: s
- List values: $[]$ (empty list), $v_1 :: v_2$ (cons)
- Tuple values: $(v_1, v_2, \ldots, v_n)$
- Closure values: $\langle \texttt{fn}\ p\ \texttt{->}\ e, \rho \rangle$ representing a function with parameter pattern p, body e, and captured environment ρ
- Constructor values: $C(v)$ representing algebraic data type constructors applied to values

Environments: ρ denotes value environments, which are finite maps from variables to values, written as $\{x_1 \mapsto v_1, \ldots, x_n \mapsto v_n\}$. We write:

- $\rho(x)$ for environment lookup, returning the value bound to x
- $\rho[x \mapsto v]$ for environment extension, binding x to v
- $\emptyset$ for the empty environment
- $\rho_1 + \rho_2$ for environment union (right-biased: ρ_2 shadows ρ_1)

Evaluation Judgment: The primary judgment form is:

$$\langle e, \rho \rangle \Downarrow v$$

read as "expression e evaluates to value v in environment ρ."

Pattern Matching: Pattern matching produces bindings from patterns and values. We write:

$$\texttt{match}(p, v) = \rho$$

to indicate that pattern p successfully matches value v, producing environment ρ containing the bindings. If matching fails, we write $\texttt{match}(p, v) = \bot$ (failure).

Auxiliary Notation:

- $\{e_i\}_{i=1}^n$ denotes a sequence of expressions $e_1, \ldots, e_n$
- $\{v_i\}_{i=1}^n$ denotes a sequence of values $v_1, \ldots, v_n$
- Inference rules are written with premises above the line and conclusion below
- Side conditions are written to the right of rules when necessary

2) Evaluation Rules: We now present the complete evaluation rules for LambdaScript expressions.

Literals:

$$\frac{}{\langle n, \rho \rangle \Downarrow n} \quad \frac{}{\langle b, \rho \rangle \Downarrow b} \quad \frac{}{\langle s, \rho \rangle \Downarrow s} \quad \frac{}{\langle (), \rho \rangle \Downarrow ()}$$

Variables:

$$\frac{x \mapsto v \in \rho}{\langle x, \rho \rangle \Downarrow v}$$

Functions (Closures):

$$\frac{}{\langle \texttt{fn}\ p\ \texttt{->}\ e, \rho \rangle \Downarrow \langle \texttt{fn}\ p\ \texttt{->}\ e, \rho \rangle}$$

A function expression evaluates to a closure capturing the current environment.

Application:

$$\frac{\langle e_1, \rho \rangle \Downarrow \langle \texttt{fn}\ p\ \texttt{->}\ e, \rho' \rangle \quad \langle e_2, \rho \rangle \Downarrow v_2 \quad \texttt{match}(p, v_2) = \rho'' \quad \langle e, \rho' + \rho'' \rangle \Downarrow v}{\langle e_1\ e_2, \rho \rangle \Downarrow v}$$

Application evaluates the function to a closure, evaluates the argument, matches the argument against the parameter pattern, and evaluates the body in the extended environment.

Let Binding:

$$\frac{\langle e_1, \rho \rangle \Downarrow v_1 \quad \texttt{match}(p, v_1) = \rho' \quad \langle e_2, \rho + \rho' \rangle \Downarrow v_2}{\langle \texttt{let}\ p = e_1\ \texttt{in}\ e_2, \rho \rangle \Downarrow v_2}$$

Recursive Let Binding:

$$\frac{\langle e_1, \rho[x \mapsto \langle \texttt{fn}\ p\ \texttt{->}\ e_1, \rho[x \mapsto \cdots]\rangle]\rangle \Downarrow v_1 \quad \langle e_2, \rho[x \mapsto v_1]\rangle \Downarrow v_2}{\langle \texttt{let rec}\ x\ p = e_1\ \texttt{in}\ e_2, \rho\rangle \Downarrow v_2}$$

The recursive binding creates a cyclic environment where x is bound to a closure that can refer to itself.

Conditional:

$$\frac{\langle e_1, \rho\rangle \Downarrow \texttt{true} \quad \langle e_2, \rho\rangle \Downarrow v}{\langle \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3, \rho\rangle \Downarrow v}$$

$$\frac{\langle e_1, \rho\rangle \Downarrow \texttt{false} \quad \langle e_3, \rho\rangle \Downarrow v}{\langle \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3, \rho\rangle \Downarrow v}$$

Pattern Matching:

$$\frac{\langle e, \rho\rangle \Downarrow v \quad \texttt{match}(p_i, v) = \rho' \quad \langle e_i, \rho + \rho'\rangle \Downarrow v'}{\langle \texttt{case}\ e\ \texttt{do}\ \{p_j \to e_j\}_{j=1}^{n}, \rho\rangle \Downarrow v'}$$

where i is the first index such that $\texttt{match}(p_i, v) \neq \bot$.

Lists:

$$\frac{}{\langle [], \rho\rangle \Downarrow []} \qquad \frac{\langle e_1, \rho\rangle \Downarrow v_1 \quad \langle e_2, \rho\rangle \Downarrow v_2}{\langle e_1 :: e_2, \rho\rangle \Downarrow v_1 :: v_2}$$

Tuples:

$$\frac{\langle e_1, \rho\rangle \Downarrow v_1 \quad \cdots \quad \langle e_n, \rho\rangle \Downarrow v_n}{\langle (e_1, \ldots, e_n), \rho\rangle \Downarrow (v_1, \ldots, v_n)}$$

Constructors:

$$\frac{}{\langle C, \rho\rangle \Downarrow C} \qquad \frac{\langle e, \rho\rangle \Downarrow v}{\langle C\ e, \rho\rangle \Downarrow C(v)}$$

Binary Operators:

$$\frac{\langle e_1, \rho\rangle \Downarrow n_1 \quad \langle e_2, \rho\rangle \Downarrow n_2 \quad n_3 = n_1 \oplus n_2}{\langle e_1\ \texttt{op}\ e_2, \rho\rangle \Downarrow n_3}$$

where $\oplus$ denotes the semantic interpretation of the operator.

3) Pattern Matching Rules: The auxiliary function $\texttt{match}(p, v)$ attempts to match pattern p against value v, returning an environment of bindings or failure ($\bot$).

Variable Pattern:

$$\texttt{match}(x, v) = \{x \mapsto v\}$$

Wildcard Pattern:

$$\texttt{match}(\_, v) = \emptyset$$

Literal Patterns:

$$\texttt{match}(n, n) = \emptyset \quad \texttt{match}(b, b) = \emptyset \quad \texttt{match}((), ()) = \emptyset$$

If the literal values don't match: $\texttt{match}(n, n') = \bot$ when $n \neq n'$.

List Patterns:

$$\texttt{match}([], []) = \emptyset$$

$$\texttt{match}(p_1 :: p_2, v_1 :: v_2) = \texttt{match}(p_1, v_1) + \texttt{match}(p_2, v_2)$$

if both sub-matches succeed; otherwise $\bot$.

Tuple Patterns:

$$\texttt{match}((p_1, \ldots, p_n), (v_1, \ldots, v_n)) = \texttt{match}(p_1, v_1) + \cdots + \texttt{match}(p_n, v_n)$$

if all sub-matches succeed; otherwise $\bot$.

Constructor Patterns:

$$\texttt{match}(C, C) = \emptyset$$

$$\texttt{match}(C\ p, C(v)) = \texttt{match}(p, v)$$

If constructors don't match: $\texttt{match}(C, C') = \bot$ when $C \neq C'$.

4) Evaluation Examples: **Example 1: Function Application**

Consider evaluating `(fn x -> x + 1) 5` in empty environment $\emptyset$:

1) $\langle \texttt{fn}\ x \texttt{ -> } x + 1, \emptyset \rangle \Downarrow \langle \texttt{fn}\ x \texttt{ -> } x + 1, \emptyset \rangle$
2) $\langle 5, \emptyset \rangle \Downarrow 5$
3) $\texttt{match}(x, 5) = \{x \mapsto 5\}$
4) $\langle x + 1, \{x \mapsto 5\} \rangle \Downarrow 6$
5) Therefore: $\langle (\texttt{fn}\ x \texttt{ -> } x + 1)\ 5, \emptyset \rangle \Downarrow 6$

Example 2: Pattern Matching on Lists

Evaluating `case [1, 2] do | [] -> 0 | h :: t -> h` in $\emptyset$:

1) $\langle [1, 2], \emptyset \rangle \Downarrow 1 :: (2 :: [])$
2) First pattern $[]$ fails: $\texttt{match}([], 1 :: (2 :: [])) = \bot$
3) Second pattern $h :: t$ succeeds: $\texttt{match}(h :: t, 1 :: (2 :: [])) = \{h \mapsto 1, t \mapsto 2 :: []\}$
4) $\langle h, \{h \mapsto 1, t \mapsto 2 :: []\} \rangle \Downarrow 1$
5) Result: 1

These operational semantics define the complete runtime behavior of LambdaScript and provide a formal foundation for reasoning about program correctness.

VI. Implementation

The LambdaScript interpreter is implemented in OCaml and consists of a multi-pass pipeline that transforms source code into typed, evaluated results. This section describes the architecture and implementation details of each component.

A. Architecture Overview

The interpreter follows a traditional compiler pipeline architecture with the following phases:

1) **Lexical Analysis (Lexer)**: Tokenizes source code into a stream of tokens
2) **Syntactic Analysis (Parser)**: Constructs an abstract syntax tree (AST) from tokens using parser combinators
3) **AST Simplification**: Transforms the parser AST into a condensed representation suitable for type checking
4) **Type Inference**: Performs constraint-based type inference using Algorithm W
5) **Type Fixing**: Resolves recursive types and applies type substitutions throughout the AST
6) **Evaluation**: Interprets the typed AST using environment-based semantics

The implementation uses two distinct AST representations:

- **Parser AST** (`Expr.expr`): A surface-level representation that closely mirrors the concrete syntax, preserving operator precedence information and syntactic structure
- **Core AST** (`Cexpr.c_expr`): A simplified representation optimized for type checking and evaluation, with explicit operator applications and normalized structure

This two-stage approach separates parsing concerns from semantic analysis, allowing the parser to focus on syntactic correctness while the core representation facilitates type inference and evaluation.

B. Lexer and Parser

1) Lexical Analysis: The lexer (`lex.ml`) implements a token-based scanner that transforms source code into a sequence of tokens. Tokens represent atomic lexical units such as keywords, identifiers, literals, and operators.

Token types include:

- **Literals**: `Integer`, `FloatToken`, `Boolean`, `CharToken`, `StringToken`
- **Keywords**: `Let`, `Rec`, `And`, `In`, `If`, `Then`, `Else`, `Case`, `Do`, `Fn`, `Type`
- **Identifiers**: `Id` (lowercase), `TypeVar` (starts with `'`), type constructors
- **Operators**: `Plus`, `Minus`, `Times`, `Divide`, `Mod`, `AND`, `OR`, `ConsToken` (`::`)
- **Comparison**: `EQ`, `NE`, `LT`, `GT`, `LE`, `GE`
- **Delimiters**: `LParen`, `RParen`, `LBracket`, `RBracket`, `LBrace`, `RBrace`
- **Custom operators**: `Relop`, `Addop`, `Mulop`, `Logop` for user-defined operators

The lexer maintains line number information for each token, enabling precise error reporting. String and character literals support escape sequences (`\n`, `\t`, `\"`, etc.). Comments are handled using OCaml-style syntax: `(* ... *)`.

2) Parser Combinators: The parser (`parser.ml`) uses a combinator-based approach, defining a parser as a function from token streams to optional parse results:

```
type 'a parser =
  token_type list -> ('a * token_type list) option
```

Parser combinators provide compositional building blocks:

- `<|>` (choice): Tries first parser; if it fails, tries second parser
- `>>` (sequence): Chains parsers, passing results forward
- `*>` (right): Sequences two parsers, keeping right result
- `<*` (left): Sequences two parsers, keeping left result
- `>>=` (bind): Monadic bind for dependent parsing
- `let*` (binding operator): Syntactic sugar for `>>=`, enabling clean monadic code

Example combinator usage:

```
(* Parse "if e1 then e2 else e3" *)
let if_parser =
  let* _ = expect_token If in
  let* e1 = expr_parser in
  let* _ = expect_token Then in
  let* e2 = expr_parser in
  let* _ = expect_token Else in
  let* e3 = expr_parser in
  return (Ternary (e1, e2, e3))
```

The parser handles operator precedence through a stratified grammar with separate parsing levels for disjunction (`||`), conjunction (`&&`), comparison, cons (`::`), addition, multiplication, and atomic expressions. This ensures correct precedence without explicit precedence tables.

Pattern parsing supports all pattern forms including literals, variables, wildcards, cons patterns, tuples, and constructor applications. Type annotations are parsed alongside expressions and patterns, allowing optional type information at function parameters and let bindings.

C. Type Inference Engine

The type inference engine (`typecheck.ml`) implements a constraint-based variant of Algorithm W, the classical Hindley-Milner type inference algorithm. Type inference proceeds in three phases: constraint generation, unification, and generalization.

1) Constraint Generation: Given an expression and a typing environment, constraint generation produces a type (possibly containing fresh type variables) and a set of type equations that must be satisfied for the expression to be well-typed.

The constraint generation function has type:

```
generate : type_env -> c_expr ->
  (mono_type * type_equations)
```

Fresh type variables are generated using a counter-based approach, ensuring unique identifiers for each unknown type. Constraints are accumulated as the expression tree is traversed.

Example constraint generation:

For function application `e1 e2`:

1) Generate constraints for `e1`, obtaining type τ_1 and constraints C_1
2) Generate constraints for `e2`, obtaining type τ_2 and constraints C_2
3) Create fresh type variable α for the result
4) Add constraint $\tau_1 = \tau_2 \to \alpha$
5) Return type α with constraints $C_1 \cup C_2 \cup \{(\tau_1, \tau_2 \to \alpha)\}$

Let-bound variables require special handling to support polymorphism. After inferring the type of the bound expression, we solve its constraints, then generalize the resulting type before adding it to the environment for the body. This implements let-polymorphism, allowing expressions like:

```
let id = fn x -> x in
(id 5, id "hello")
```

where `id` is used at both `int -> int` and `string -> string`.

2) Unification: The unification algorithm solves a set of type equations by computing a most general unifier (MGU)—a substitution that, when applied to both sides of each equation, makes them equal.

Unification rules:

- $\mathcal{U}(\alpha, \tau)$: If α does not occur in τ, return $[\tau/\alpha]$; otherwise fail (occurs check)
- $\mathcal{U}(\tau, \alpha)$: Symmetric to above
- $\mathcal{U}(\tau_1 \to \tau_2, \tau_1' \to \tau_2')$: Recursively unify τ_1 with τ_1' and τ_2 with τ_2'
- $\mathcal{U}([\tau], [\tau'])$: Recursively unify τ with τ'
- $\mathcal{U}((\tau_1, \ldots, \tau_n), (\tau_1', \ldots, \tau_n'))$: Recursively unify each pair
- $\mathcal{U}(T\langle\tau_1, \ldots, \tau_n\rangle, T\langle\tau_1', \ldots, \tau_n'\rangle)$: Unify type applications

The occurs check prevents infinite types by ensuring a type variable does not appear in its own definition. For recursive types, we use explicit fixed-point type constructors (`FixedPoint`) rather than allowing infinite unfolding.

Type errors are reported when:

- A type variable occurs in its substituted type (occurs check failure)
- Two primitive types are equated (e.g., `int = bool`)
- Type constructors have different arities or names
- Patterns and scrutinees have incompatible types

3) Generalization and Instantiation: After constraint solving, types are generalized by introducing universal quantifiers for free type variables. The generalization function:

$$\text{gen}(\Gamma, \tau) = \forall\{\alpha_1, \ldots, \alpha_n\}.\tau$$

where $\{\alpha_1, \ldots, \alpha_n\} = FV(\tau) \setminus FV(\Gamma)$.

Type schemes are instantiated by replacing quantified variables with fresh type variables:

$$\text{inst}(\forall\alpha.\sigma) = \sigma[\beta/\alpha]$$

This allows polymorphic functions to be used at multiple types while maintaining type safety.

4) Type Fixing: After type inference, the type fixer (`typefixer.ml`) applies the computed substitution throughout the entire AST, replacing type variables with their inferred types. This phase also:

- Resolves recursive type references using fixed-point constructors
- Validates that all type variables have been instantiated
- Ensures mutually recursive types reference each other correctly
- Annotates the AST with complete type information for evaluation

D. Pattern Matching

Pattern matching in LambdaScript supports comprehensive destructuring with exhaustiveness checking and redundancy detection.

1) Pattern Compilation: Patterns are compiled into decision trees that efficiently test values against pattern structure. The compilation process:

1) **Flattening**: Nested patterns are flattened into sequences of simple tests
2) **Grouping**: Patterns with the same constructor are grouped together
3) **Optimization**: Redundant tests are eliminated

Pattern matching executes by traversing the decision tree, testing each pattern in order until a match is found. Variable bindings are collected during matching and added to the evaluation environment.

2) Exhaustiveness Checking: The exhaustiveness checker (`exhaustiveness.ml`) ensures that pattern matches cover all possible values. This analysis prevents runtime match failures by detecting missing cases at compile time.

The algorithm constructs a symbolic representation of the input space and removes the portion covered by each pattern. If any portion remains uncovered, the match is non-exhaustive and an error is reported.

Example exhaustiveness violations:

```
(* Non-exhaustive: missing Nil case *)
case lst do
| h :: t -> h

(* Exhaustive: all cases covered *)
case lst do
| [] -> 0
| h :: t -> h
```

For user-defined algebraic data types, exhaustiveness checking verifies that all constructors are matched. Wildcard patterns and variable patterns match any value, making the entire match exhaustive if they appear as the final case.

E. Evaluator

The evaluator (`ceval.ml`) implements the operational semantics using environment-based interpretation with closures.

1) Value Representation: Values are the results of evaluation:

```
type value =
  | IntVal of int
  | BoolVal of bool
  | StringVal of string
  | CharVal of char
  | UnitVal
  | ListVal of value list
  | TupleVal of value list
  | ClosureVal of c_pat * c_expr * env
  | ConstructorVal of string * value option
```

Closures capture their defining environment, enabling proper handling of free variables and supporting higher-order functions with lexical scoping.

2) Environment-Based Evaluation: Environments map variable names to values. Evaluation judgments have the form:

$$\langle e, \rho \rangle \Downarrow v$$

The evaluator recursively descends the expression tree, accumulating bindings in the environment and producing values.

Key evaluation rules:

- **Variables**: Look up in environment, fail if unbound
- **Functions**: Create closure capturing current environment
- **Application**: Evaluate function to closure, evaluate argument, match pattern against argument value, evaluate body in extended environment

- **Let bindings**: Evaluate bound expression, match pattern, extend environment for body
- **Recursive let**: Create cyclic environment where bound variable refers to itself
- **Pattern matching**: Evaluate scrutinee, try each branch in order until pattern matches

3) Built-in Functions: Built-in functions are implemented as primitive operations in the evaluator:

- **I/O**: `print`, `println`
- **Conversions**: `int_to_str`, `int_to_float`, `float_to_int`, `string_to_list`
- **Higher-order**: `map`, `filter`, `reduce_left`, `reduce_right`

These functions are pre-populated in the initial environment with appropriate types and implementations.

F. Advanced Features

1) Mutually Recursive Types: Mutually recursive type definitions are handled by:

1) Adding all type names to the environment simultaneously
2) Type-checking constructor payloads with all names in scope
3) Using fixed-point type constructors to represent cycles
4) Verifying that cycles are guarded by constructors (preventing infinite types)

2) List Comprehensions: List comprehensions desugar into nested `map` and `filter` operations:

```
[x * 2 | x => lst, x > 5]
```

desugars to:

```
map (fn x -> x * 2)
    (filter (fn x -> x > 5) lst)
```

3) Custom Operators: User-defined operators are supported through operator definitions. Operators are first-class values when wrapped in parentheses:

```
let (++) = fn a -> fn b -> a + b in
(++) 3 4  (* Returns 7 *)
```

Custom operators follow the precedence of their first character (+, −, ⋆, /, etc.).

4) Code Blocks: Code blocks provide sequential execution with local definitions:

```
{
  let x = 5;
  let y = 10;
  x + y
}
```

Blocks desugar into nested let expressions, maintaining proper scoping of local bindings.

VII. Testing and Validation

A. Test Suite Design

B. Coverage Analysis

C. Real-World Examples

VIII. Discussion

A. Known Limitations

B. Related Work

C. Future Work

IX. Conclusion